Diego Enrique Morales Zepeda

Resultados y análisis del experimento EXP-007

Revocación efectiva del consentimiento de datos financieros (Open Finance)
(HU3.2.1)

Hipótesis evaluada

Si se implementa en el microservicio de Perfilamiento la validación del consentimiento vigente contra el microservicio de Consentimiento Open Finance antes de cada consulta de datos financieros, con una caché local de TTL de 60 s, entonces, cuando un cliente revoca su consentimiento en operación normal (10 TPS), Perfilamiento deja de entregar sus datos (responde 403) en 5 minutos o menos en el 100 % de los casos, sin conceder ningún acceso con el consentimiento revocado después de ese plazo, sin rechazar a clientes con consentimiento vigente y con un p95 de 300 ms o menos en la consulta, y cada revocación queda registrada en el audit_log. El experimento responde al escenario de calidad QS-015 (seguridad) de la HU3.2.1.

Se ejecutó una corrida en AWS Academy (región us-east-1) el 12 de noviembre de 2026, entre las 16:22 y las 16:47 UTC, siguiendo el procedimiento diseñado:

- **Carga de fondo:** JMeter envió 10 TPS durante 25 minutos a `GET /perfiles/{clienteId}` con 50 clientes que nunca se revocaron. Cualquier 403 en esa carga cuenta como rechazo falso.
- **Medición:** en paralelo, un script en Python tomó otros 50 clientes con consentimiento vigente y, uno cada 5 s, (a) consultó el perfil y verificó 200; (b) revocó el consentimiento y registró t0 cuando Consentimiento lo confirmó; (c) consultó Perfilamiento cada 2 s hasta el primer 403 (t1); y (d) siguió consultando cada 10 s durante 6 minutos para verificar que no volviera a aparecer un 200.
- **Infraestructura:** Consentimiento, Perfilamiento y el mock de Open Finance en ECS Fargate (una tarea por servicio) detrás de un balanceador con HTTPS, y una base PostgreSQL cifrada por servicio en RDS. En Perfilamiento, `CONSENT_CHECK_ENABLED=true` y `OPEN_FINANCE_ENABLED=true`; los flags de los demás experimentos, apagados.

1. Resultados del experimento

1.1 Criterio de aceptación

Tabla 1. Resultados frente al criterio de aceptación del diseño. La hipótesis se refuta si cualquiera de las condiciones falla.

Condición	Meta	Resultado	¿Cumple?
Δt máximo de revocación (50 revocaciones)	≤ 300 s	61,98 s	Sí
Accesos concedidos con consentimiento revocado después de 5 min	0	0	Sí
Rechazos a clientes con consentimiento vigente	0	0 (0 de 15.014 en JMeter; 0 de 50 en el script)	Sí
Latencia p95 de GET /perfiles (JMeter, 25 min)	≤ 300 ms	85 ms	Sí
Revocaciones registradas en el audit_log	50/50	50/50	Sí

Las cinco condiciones se cumplen, de modo que **la hipótesis se confirma**. Las secciones siguientes detallan cada resultado y un hallazgo sobre la latencia que no afecta el criterio, pero que se analiza en la sección 2.5.

1.2 Tiempo de revocación efectiva

Tabla 2. Δt = t1 − t0 (tiempo entre la confirmación de la revocación y el primer 403). La segunda columna excluye al cliente 1, explicado en la sección 2.1.

Estadístico	50 clientes	49 clientes (sin el cliente 1)
Mínimo	33,27 s	59,96 s
Promedio	59,79 s	60,33 s
Mediana	60,19 s	60,20 s
Percentil 95	61,97 s	61,97 s
Máximo	61,98 s	61,98 s

De las 50 revocaciones, 43 dieron un Δt entre 59,9 y 60,5 s, 2 entre 60,5 y 61,5 s y 4 cerca de 62 s; la restante (el cliente 1) dio 33,27 s (Figura 1). Ningún valor se acercó al límite de 300 s.

Mientras la entrada en caché no expiraba, Perfilamiento siguió respondiendo 200 a los clientes ya revocados: 1.441 respuestas en total, unas 29 por cliente con el sondeo cada 2 s, todas dentro de los primeros 62 s. Después del primer 403 de cada cliente no volvió a aparecer ningún 200 durante los 6 minutos de vigilancia, no hubo otras respuestas (errores o tiempos agotados) y el mock de Open Finance no recibió ninguna llamada para ese cliente. Las 50 revocaciones quedaron registradas en el audit_log.

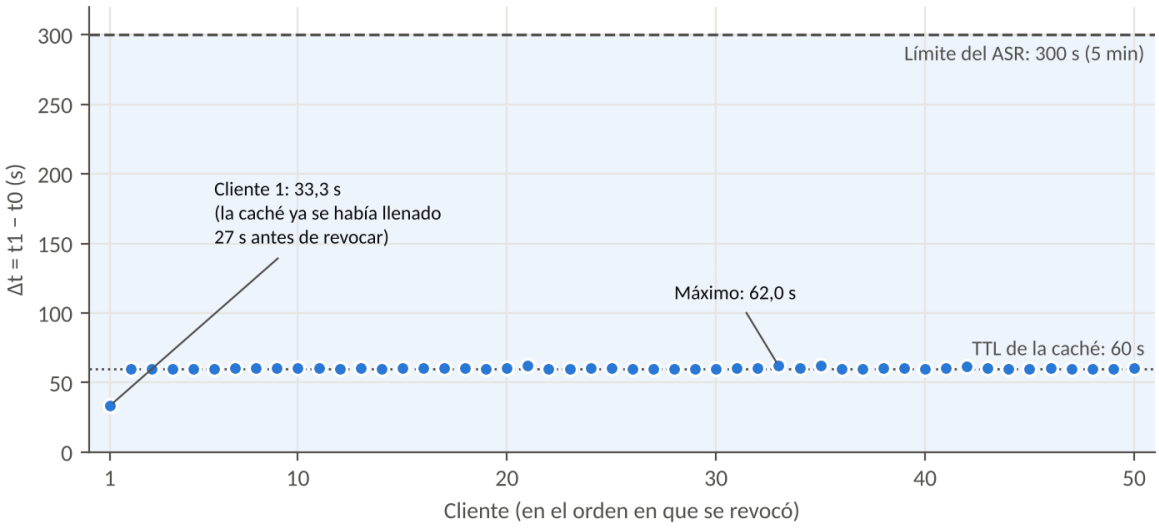


Figura 1. Δt de cada revocación, en el orden en que se revocaron. Todos los clientes quedan alrededor del TTL de 60 s y muy por debajo del límite de 300 s.

1.3 Carga, rechazos falsos y latencia

Tabla 3. Resultados de la carga de fondo de JMeter.

Métrica	Valor
Solicitudes	15.014
Duración	25 min (16:22:30 – 16:47:30 UTC)
Throughput	10,01 TPS
Respuestas 200 / errores	15.014 (100 %) / 0
Latencia p50 / p90 / p95 / p99	64 / 77 / 85 / 255 ms
Latencia máxima	796 ms
Solicitudes por encima de 300 ms	111 (0,74 %)

Tabla 4. Latencia de JMeter por fase de la prueba.

Fase	Intervalo (UTC)	Solicitudes	p50	p95	p99	Máx.
Arranque	16:22:30 – 16:23:31	617	64 ms	96 ms	315 ms	478 ms
Medición (revocaciones y vigilancia)	16:23:31 – 16:34:37	6.660	64 ms	88 ms	334 ms	796 ms
Después de la medición	16:34:37 – 16:47:30	7.737	63 ms	84 ms	186 ms	546 ms

El throughput se mantuvo en 10 TPS durante toda la prueba y ninguna de las 15.014 solicitudes a clientes vigentes fue rechazada. La mediana se mantuvo entre 61 y 67 ms en todas las ventanas de 30 s (Figura 2).

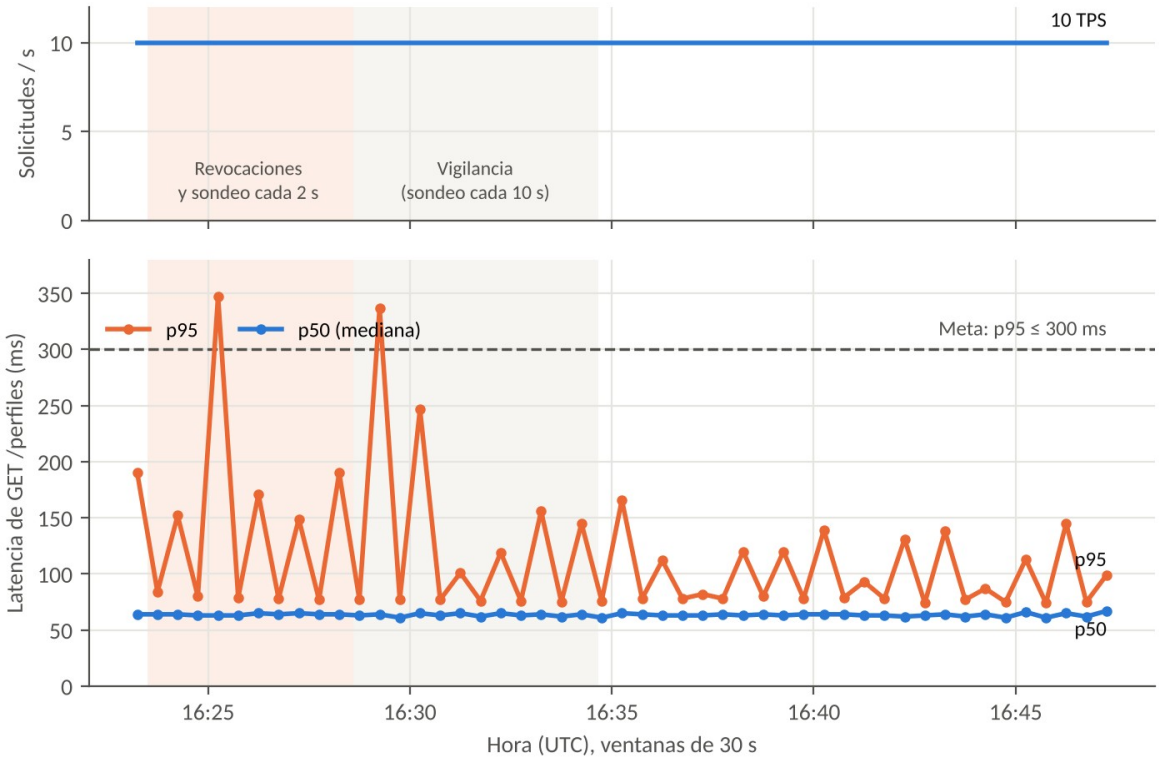


Figura 2. Arriba, solicitudes por segundo de JMeter; abajo, p50 y p95 por ventana de 30 s. La franja naranja marca las revocaciones con sondeo cada 2 s y la gris la vigilancia. Se omiten la primera y la última ventana, que están incompletas.

1.4 Picos periódicos de latencia

El p95 por ventana de 30 s sube y baja con un periodo de un minuto (Figura 2), y dos ventanas superaron los 300 ms: las que empiezan a las 16:25:00 (347 ms) y a las 16:29:00 (337 ms). Al agrupar las solicitudes por el segundo del reloj en que empezaron (Figura 3) se ve que **110 de las 111 solicitudes de más de 300 ms empezaron entre los segundos :01 y :04 de algún minuto**. Esos 4 segundos concentran 1.000 solicitudes (6,7 %), con p50 de 102 ms y p95 de 393 ms. En las otras 14.014 solicitudes el p50 fue de 63 ms, el p95 de 78 ms y el p99 de 85 ms, y solo una superó los 300 ms: la primera de la prueba (459 ms, al abrir las conexiones).

El pico fue mayor durante la medición que después: en los segundos :01 a :04, el p95 fue de 509 ms entre las 16:23:31 y las 16:34:37 y de 277 ms entre las 16:34:37 y el final. Fuera de esos segundos, el p95 fue el mismo en ambas fases.

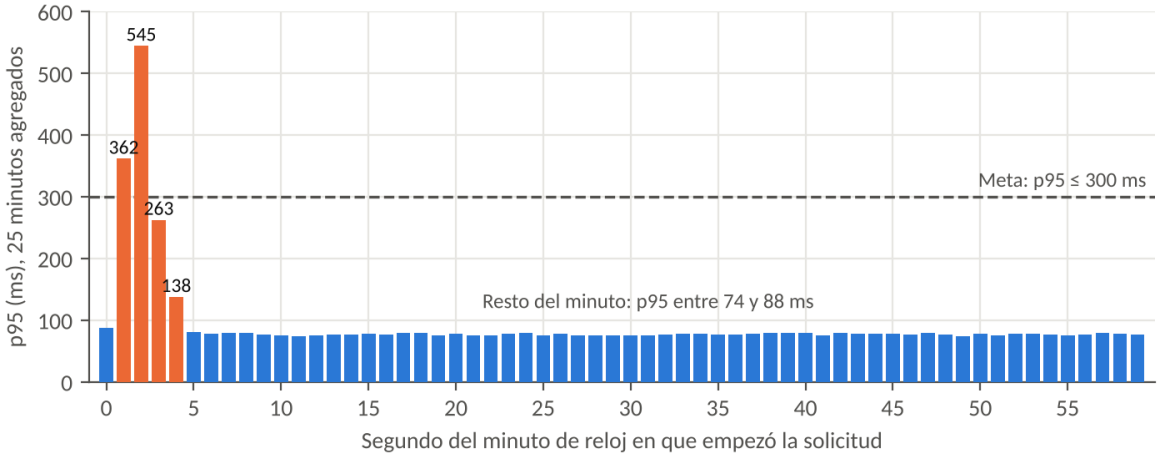


Figura 3. p95 de JMeter según el segundo del minuto de reloj en que empezó cada solicitud, agregando los 25 minutos. Casi toda la cola de latencia se concentra en los segundos :01 a :04.

2. Análisis de resultados

2.1 La caché con TTL acota el tiempo de revocación

El Δt de cada cliente depende de cuánto tiempo le quedaba a su entrada en la caché de Perfilamiento en el momento de la revocación: mientras la entrada es válida, Perfilamiento responde con el valor guardado («vigente»); cuando expira, vuelve a preguntar a Consentimiento, recibe «no vigente» y empieza a responder 403. Por eso Δt queda entre 0 y el TTL, más la resolución del sondeo.

El procedimiento mide deliberadamente el **peor caso**: en el paso (a) el script consulta el perfil justo antes de revocar, lo que deja la entrada recién creada. Así, 49 de los 50 clientes dieron un Δt de 60 s, el TTL completo. Los 4 valores cercanos a 62 s se explican por la resolución de 2 s del sondeo (el 403 apareció entre dos consultas), y los que quedan unos milisegundos por debajo de 60 s, porque la entrada se crea en el paso (a) unos milisegundos antes de que el script registre t0, que se toma cuando Consentimiento confirma la revocación.

El cliente 1 confirma el mecanismo. Su perfil se había consultado a mano 27 s antes de la revocación (para mostrarlo en el video), de modo que su entrada ya estaba creada cuando el script lo revocó y solo le quedaban unos 33 s de vida: su Δt fue de 33,27 s y recibió 16 respuestas 200 en lugar de 29. No es un error de medición sino el comportamiento esperado cuando la entrada no está recién creada, y el criterio se cumple con o sin ese cliente.

La cota depende del TTL y no de la carga: las 50 revocaciones, hechas bajo 10 TPS, dieron el mismo Δt. El peor valor medido (62 s) queda casi 5 veces por debajo del límite del ASR (300 s). Ese margen convierte al TTL en el parámetro de diseño: subirlo reduce las consultas a Consentimiento (con 50 clientes a 10 TPS cada cliente se consulta cada 5 s, así que con TTL de 60 s solo una de cada 12 consultas llega a Consentimiento, según el diseño de la carga) a costa de una revocación más lenta, y bajarlo hace lo contrario. Cualquier TTL de hasta unos 295 s seguiría cumpliendo el ASR.

Hay que precisar que, durante esa ventana de hasta 60 s, Perfilamiento sigue entregando los datos del cliente que ya revocó (1.441 respuestas en esta prueba). El ASR lo permite, porque exige dejar de usar los datos en 5 minutos o menos, pero significa que la revocación no es inmediata.

2.2 La verificación ocurre antes de pedir los datos

Que el mock de Open Finance no recibiera ninguna llamada después del primer 403 de cada cliente demuestra que la verificación del consentimiento corre antes de cualquier acceso a los datos financieros: en `main.py` de Perfilamiento, `verificar_acceso` es una dependencia del endpoint y se ejecuta antes de su cuerpo, que es donde se llama a Open Finance. Tampoco volvió a aparecer un 200 después del primer 403: la respuesta «no vigente» también se guarda en la caché y se respeta.

El comportamiento fail-closed (negar el acceso si Consentimiento no responde) no se ejercitó en esta corrida, porque Consentimiento estuvo disponible todo el tiempo. Se verificó en la prueba de integración local, deteniendo Consentimiento: Perfilamiento respondió 403 con el motivo «no verificable».

2.3 La auditoría queda completa

Las 50 revocaciones quedaron en el audit_log porque el cambio de estado y su registro se guardan en la misma transacción de la base de Consentimiento: Consentimiento solo confirma la revocación (el t0 del experimento) cuando ambos están guardados, así que no puede existir una revocación sin registro. Además, la revocación es idempotente: revocar dos veces no cambia nada ni duplica el registro.

2.4 Costo de la táctica

La validación tiene un costo bajo. La mediana fue de 63 a 64 ms en todas las fases, y de ese tiempo unos 50 ms corresponden a la latencia que simula el mock de Open Finance, de modo que la verificación del JWT, la consulta del consentimiento (casi siempre en caché), la base de datos y la red suman del orden de 15 ms a la mediana. Fuera de los picos de la sección 1.4, el p95 fue de 78 ms y no cambió entre la fase de revocaciones y la posterior, es decir, las revocaciones y las consultas a Consentimiento al expirar la caché no movieron la latencia de forma apreciable. Tampoco hubo rechazos falsos.

2.5 Los picos periódicos no se deben a la táctica

Los picos de latencia de los segundos :01 a :04 de cada minuto no se explican por la caché de consentimiento, por tres razones:

- Están alineados con el reloj y no con la caché. Las entradas de los clientes de fondo se crearon en los primeros 5 s de la prueba (16:22:31 a 16:22:35), así que expiran y se renuevan hacia los segundos :31 a :35 de cada minuto, donde la latencia es normal.
- Aparecen desde el primer minuto (16:23:01), antes de que pudiera expirar cualquier entrada.
- No hay un cliente que se ponga lento una vez por minuto: en un mismo segundo afectan a solicitudes de clientes distintos.

Todo apunta a un proceso periódico de la infraestructura que compite por recursos una vez por minuto (en la instancia que genera la carga, en la tarea de Perfilamiento de 0,5 vCPU o en la base db.t3.micro). Su efecto creció durante la medición (p95 de 509 ms en el pico, frente a 277 ms después), cuando el script sumaba aproximadamente entre 6 y 11 consultas por segundo a los 10 TPS de JMeter. La causa no se pudo determinar porque no se recogieron métricas de CPU por segundo de cada componente.

El criterio del experimento se define sobre el p95 de toda la prueba (85 ms) y se cumple con amplitud. Sin embargo, si se evaluara por ventanas de 30 s, dos de las 49 ventanas lo incumplirían. Conviene investigarlo antes de asumir garantías de latencia por ventana corta.

2.6 Validez y limitaciones

- Se hizo una sola corrida con 50 revocaciones. No hubo corrida de control con la validación apagada porque el diseño no la incluía; en esa configuración Perfilamiento nunca consulta a Consentimiento, así que la revocación no tendría ningún efecto.
- La resolución del sondeo es de 2 s hasta el primer 403 y de 10 s en la vigilancia, de modo que Δt puede estar sobrestimado hasta en 2 s.
- Las consultas del script se suman a la carga de JMeter durante la medición; la Tabla 4 separa ambas fases.
- El certificado del balanceador es autofirmado: el tráfico va cifrado, pero los clientes no verifican la identidad del servidor (`CONSENTIMIENTO_TLS_VERIFY=false` y `--insecure`). Los JWT los emitió el script con una llave compartida (HS256), porque AWS Academy no permite usar Amazon Cognito.
- Cada servicio corrió con una sola tarea y la caché vive en la memoria del proceso. Con varias réplicas de Perfilamiento cada una tendría su propia caché: la cota de Δt seguiría siendo el TTL, pero aumentarían las consultas a Consentimiento.
- Respecto al diseño: se usaron dos instancias de RDS cifradas, una por servicio, en lugar de una sola (refuerza el patrón Database per Service); el mock de Open Finance se desplegó como servicio aparte, compartido con EXP-009; y la carga de JMeter usó un grupo de clientes distinto del de la medición, para medir los rechazos falsos sin interferir con las revocaciones.
- El experimento se hizo en AWS Academy con instancias pequeñas (Fargate de 0,5 vCPU y 1 GB, RDS db.t3.micro). Los tiempos absolutos de latencia pueden cambiar con otra configuración; el Δt, no, porque depende del TTL.

3. Conclusión

La hipótesis se confirma. Con la validación del consentimiento en Perfilamiento y una caché de TTL de 60 s, en operación normal a 10 TPS, las 50 revocaciones se hicieron efectivas en 61,98 s como máximo (60,3 s en promedio sin el cliente 1), muy por debajo del límite de 5 minutos del ASR. No hubo ningún acceso con consentimiento revocado después del primer 403 ni después de 5 minutos, ningún rechazo a clientes vigentes (0 de 15.014), el p95 de la consulta fue de 85 ms y las 50 revocaciones quedaron registradas en el audit_log.

El experimento muestra que el tiempo de revocación lo fija el TTL de la caché y no la carga, y que la combinación de verificación previa al acceso, consulta al único dueño del consentimiento y registro en la misma transacción cumple el escenario de seguridad con un costo de latencia bajo.

Como recomendaciones, se propone:

- Mantener la táctica y documentar el TTL como decisión de arquitectura, eligiendo su valor según el equilibrio entre la rapidez de la revocación y la carga sobre Consentimiento.
- Si el negocio llega a exigir una revocación casi inmediata, publicar un evento de revocación que invalide la caché de Perfilamiento, en lugar de esperar a que expire.
- Antes de escalar Perfilamiento a varias réplicas, evaluar una caché compartida o la invalidación por evento.
- Investigar los picos periódicos de latencia con métricas de CPU de cada componente y repitiendo la prueba con el generador de carga en otra instancia.

- En producción, usar un certificado público de ACM con verificación del servidor y emitir los tokens con Amazon Cognito.

Evidencias

Los datos crudos y los scripts están en el repositorio del equipo, en `docs/HU3.2.1-Diego/`: `medicion/resultados/` (`revocaciones.csv`, `resumen.json`, `resumen_jmeter.json`, `jmeter.jtl` y el reporte HTML de JMeter), `medicion/medir_revocacion.py`, `jmeter/carga_fondo.jmx` y la infraestructura en `infra/exp007.yaml`. El video de la ejecución está enlazado en Hélix, en la pestaña de resultados de EXP-007.